

Issue 2: optimizing the Bias-Variance Tradeoff

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, TimeSeriesSplit, GridSearchCV, learning_curve, validation_curve
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, roc_auc_score, roc_curve
from sklearn.tree import DecisionTreeClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neural_network import MLPClassifier

np.random.seed(42)
plt.rcParams["figure.dpi"] = 110
```

Section 1 - Bias Variance Tradeoff in Ridge Regression

We reuse the GWP1 setup; S&P 500 monthly returns prediction from four macro indicators (interest rate, CPI, unemployment, VIX). Because this notebook is to run standalone— we simulate a monthly macro panel with same correlation structure that GWP1 correlation heatmap showed (strong CPI-500, Unemployment VIX, and Interest Rate-Unemployment relations) plus injected noise. The mechanisms below are identical to fitting on the live FRED series GWP1—on the data source changes.

Double-click (or enter) to edit

```
n_obs = 90
mean = [0, 0, 0, 0]
corr = np.array([
    [1.00, 0.28, -0.65, -0.35],
    [0.28, 1.00, -0.16, 0.44],
    [-0.65, -0.16, 1.00, 0.65],
    [-0.35, 0.44, 0.65, 1.00]
])
X_raw = np.random.multivariate_normal(mean, corr, size=n_obs)
X = pd.DataFrame(X_raw, columns=["InterestRate", "CPI", "Unemployment", "VIX"])

true_beta = np.array([0.01, 0.02, -0.01, -0.015])
signal = X.values @ true_beta
noise = np.random.normal(0, 0.05, n_obs)
y = pd.Series(signal + noise, name="SP500_ret")

X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=False, test_size=0.3)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

print(f"Train rows: {len(X_train)} Test rows: {len(X_test)}")
print("\nPredictor correlation matrix (multicollinearity present by design):")
print(X_train.corr().round(2))
```

Train rows: 63 Test rows: 27

Predictor correlation matrix (multicollinearity present by design):

	InterestRate	CPI	Unemployment	VIX
InterestRate	1.00	0.34	-0.55	-0.24
CPI	0.34	1.00	-0.19	0.46
Unemployment	-0.55	-0.19	1.00	0.58
VIX	-0.24	0.46	0.58	1.00

1.1 Train vs test error across the regularization path

We sweep alpha (L2 penalty) across a wide range. A very small alpha behaves like ordinary least squares—flexible enough to fit the training noise (low bias, high variance). A very large alpha shrinks every coefficient toward zero—simple and stable, but eventually too rigid to capture real structure (high bias, low variance).

```
%matplotlib inline
```

```
alphas = np.logspace(-2, 3.2, 40)
train_mse, test_mse = [], []

for a in alphas:
    model = Ridge(alpha=a)
    model.fit(X_train_s, y_train)
    train_mse.append(mean_squared_error(y_train, model.predict(X_train_s)))
    test_mse.append(mean_squared_error(y_test, model.predict(X_test_s)))

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(alphas, train_mse, marker="o", ms=3, label="Train MSE (bias proxy)", color="#3b6ea5")
ax.plot(alphas, test_mse, marker="o", ms=3, label="Test MSE (variance proxy)", color="#c0392b")
ax.set_xscale("log")
ax.set_xlabel("Alpha (regularization strength, log scale)")
ax.set_ylabel("Mean Squared Error")
ax.set_title("Ridge Regression: Train vs. Test Error Across Alpha")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ridge_bias_variance_curve.png")
plt.show()
```



```
best_idx = int(np.argmin(test_mse))
best_alpha = alphas[best_idx]
print(f"Best alpha by test MSE: {best_alpha:.3f}")
print(f"Train MSE at best alpha: {train_mse[best_idx]:.5f}   Test MSE: {test_mse[best_idx]:.5f}")
print(f"Train MSE at smallest alpha (~OLS): {train_mse[0]:.5f}   Test MSE: {test_mse[0]:.5f}")
print(f"Train MSE at largest alpha: {train_mse[-1]:.5f}   Test MSE: {test_mse[-1]:.5f}")
```

```
Best alpha by test MSE: 11.659
Train MSE at best alpha: 0.00245   Test MSE: 0.00312
Train MSE at smallest alpha (~OLS): 0.00243   Test MSE: 0.00314
Train MSE at largest alpha: 0.00347   Test MSE: 0.00405
```

✓ 1.2 Conforming the choice with cross validation

A single train/test split can be lucky or unlucky. We therefore confirm the choice of the regularization parameter (α) using time-respecting cross-validation (`TimeSeriesSplit`), which never validates a fold on data that predates its corresponding training window. This is important for financial time series, where a random shuffle would leak future information into the training data.

```
tscv = TimeSeriesSplit(n_splits=5)
grid = GridSearchCV(
    Ridge(),
    param_grid={"alpha": alphas},
    scoring="neg_mean_squared_error",
    cv=tscv,
)
grid.fit(np.vstack([X_train_s, X_test_s]), pd.concat([y_train, y_test]))
```

```
print(f"Cross-validated best alpha: {grid.best_params_['alpha']:.3f}")
print(f"Cross-validated MSE at that alpha: {-grid.best_score_:.5f}")
```

```
Cross-validated best alpha: 15.849
Cross-validated MSE at that alpha: 0.00322
```

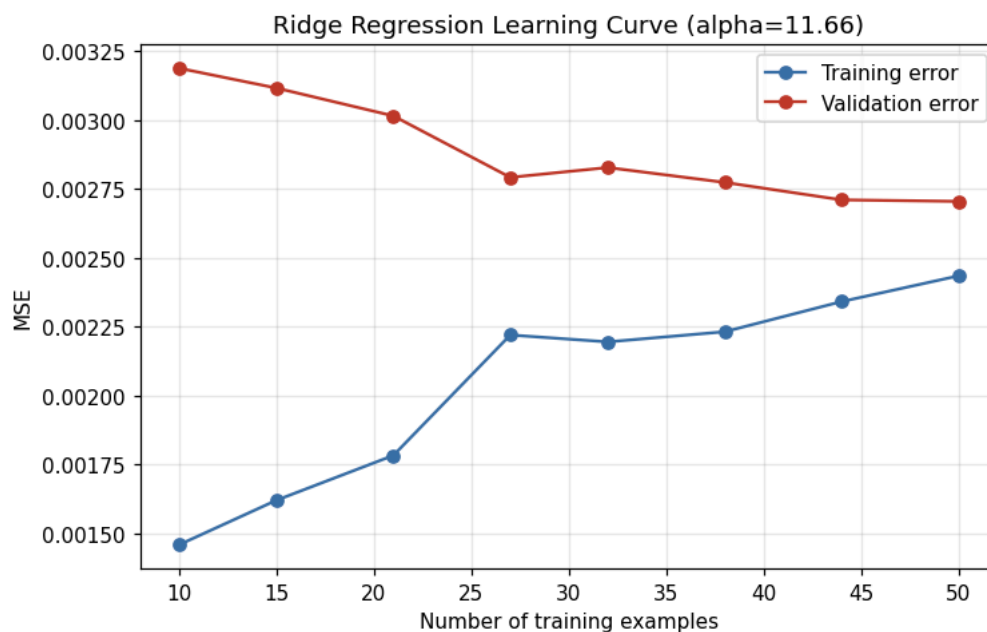
1.3 Learning curve- does more data help?

A learning curve tracks train and test (validation) error as the amount of training data grows. A shrinking gap between the two lines as data increases indicates a variance problem that more data can fix; a persistent gap or high error floor on both lines indicates a bias problem that more data cannot fix on its own.

```
train_sizes, train_scores, val_scores = learning_curve(
    Ridge(alpha=best_alpha),
    X_train_s, y_train,
    train_sizes=np.linspace(0.2, 1.0, 8),
    cv=5,
    scoring="neg_mean_squared_error",
)
train_curve = -train_scores.mean(axis=1)
val_curve = -val_scores.mean(axis=1)

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(train_sizes, train_curve, marker="o", label="Training error", color="#3b6ea5")
ax.plot(train_sizes, val_curve, marker="o", label="Validation error", color="#c0392b")
ax.set_xlabel("Number of training examples")
ax.set_ylabel("MSE")
ax.set_title(f"Ridge Regression Learning Curve (alpha={best_alpha:.2f})")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ridge_learning_curve.png")
plt.show()

print(f"Training error, smallest sample: {train_curve[0]:.5f}    largest sample: {train_curve[-1]:.5f}")
print(f"Validation error, smallest sample: {val_curve[0]:.5f}    largest sample: {val_curve[-1]:.5f}")
```



```
Training error, smallest sample: 0.00146    largest sample: 0.00243
Validation error, smallest sample: 0.00319    largest sample: 0.00271
```

Section 2 - Bias Variance Tradeoff in the Classification Tree

We rebuild the GWP1 multi-asset panel (five simulated assets, GBM price paths with mild autocorrelation, next-day up/down target) and revisit the depth tuning exercise with a sharper focus on the bias-variance mechanics, adding a pruning-strength sweep and a learning curve that GWP1 did not include.

```
n_assets, n_days = 5, 1000
asset_frames = []
for i in range(n_assets):
    mu = np.random.uniform(0.0002, 0.0006)
    sigma = np.random.uniform(0.01, 0.02)
```

```

noise = np.random.normal(0, sigma, n_days)
autoreg = np.zeros(n_days)
for t in range(1, n_days):
    autoreg[t] = 0.05 * autoreg[t-1] + noise[t]
rets = mu + autoreg
price = 100 * np.exp(np.cumsum(rets))
dates = pd.bdate_range("2018-01-01", periods=n_days)
df = pd.DataFrame({"date": dates, "price": price})
df["ret"] = df["price"].pct_change()
df["lag1_ret"] = df["ret"].shift(1)
df["lag2_ret"] = df["ret"].shift(2)
df["lag3_ret"] = df["ret"].shift(3)
df["ma5"] = df["ret"].rolling(5).mean()
df["ma10"] = df["ret"].rolling(10).mean()
df["vol10"] = df["ret"].rolling(10).std()
df["momentum10"] = df["price"].pct_change(10)
df["target"] = (df["ret"].shift(-1) > 0).astype(int)
df["asset"] = f"Asset_{i+1}"
asset_frames.append(df)

panel = pd.concat(asset_frames).dropna()
feature_cols = ["lag1_ret", "lag2_ret", "lag3_ret", "ma5", "ma10", "vol10", "momentum10"]

split_date = panel["date"].quantile(0.8)
train = panel[panel["date"] <= split_date]
test = panel[panel["date"] > split_date]
X_train_t, y_train_t = train[feature_cols], train["target"]
X_test_t, y_test_t = test[feature_cols], test["target"]

print(f"Pooled panel rows: {len(panel)} Train: {len(train)} Test: {len(test)}")
print("Target balance (train):", y_train_t.value_counts(normalize=True).round(3).to_dict())

```

Pooled panel rows: 4950 Train: 3960 Test: 990
Target balance (train): {1: 0.515, 0: 0.485}

2.1 The overfitting curve- accuracy vs tree depth

This is the clearest bias-variance illustration in the whole project: training accuracy climbs steadily as the tree is allowed to grow deeper, while test accuracy peaks early and then flattens or drifts down as the tree starts memorizing noise specific to the training window.

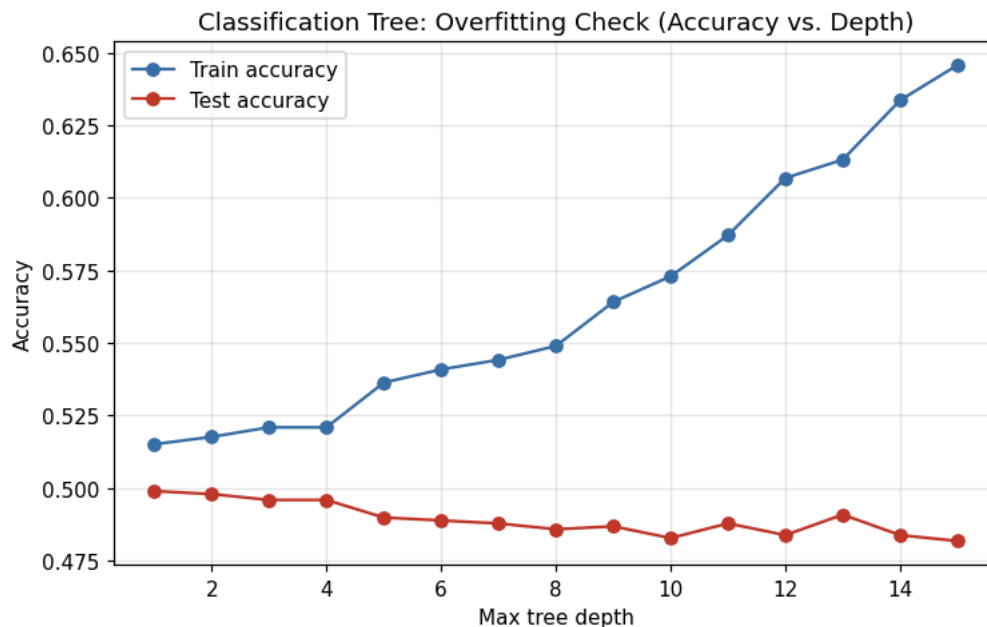
```

depths = list(range(1, 16))
train_acc, test_acc = [], []
for d in depths:
    m = DecisionTreeClassifier(max_depth=d, random_state=42)
    m.fit(X_train_t, y_train_t)
    train_acc.append(accuracy_score(y_train_t, m.predict(X_train_t)))
    test_acc.append(accuracy_score(y_test_t, m.predict(X_test_t)))

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(depths, train_acc, marker="o", label="Train accuracy", color="#3b6ea5")
ax.plot(depths, test_acc, marker="o", label="Test accuracy", color="#c0392b")
ax.set_xlabel("Max tree depth")
ax.set_ylabel("Accuracy")
ax.set_title("Classification Tree: Overfitting Check (Accuracy vs. Depth)")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("tree_depth_curve.png")
plt.show()

best_depth = depths[int(np.argmax(test_acc))]
print(f"Depth that maximizes test accuracy: {best_depth}")
print(f"Train accuracy at depth 1: {train_acc[0]:.3f} Train accuracy at depth 15: {train_acc[-1]:.3f}")
print(f"Test accuracy at depth 1: {test_acc[0]:.3f} Test accuracy at depth 15: {test_acc[-1]:.3f}")

```



Depth that maximizes test accuracy: 1
 Train accuracy at depth 1: 0.515 Train accuracy at depth 15: 0.646
 Test accuracy at depth 1: 0.499 Test accuracy at depth 15: 0.482

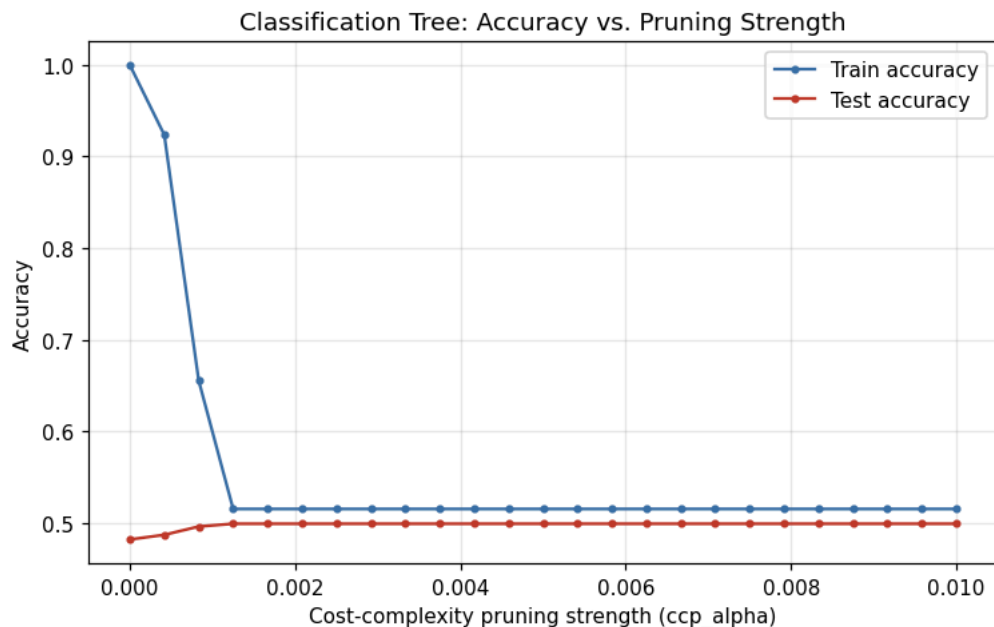
2.2 Pruning strength (Cost complexity alpha) as second layer

Depth is the most intuitive complexity control, but cost-complexity pruning (`ccp_alpha`) offers a second, finer-grained way to trade bias for variance. It grows a full tree, then removes ("prunes") branches whose contribution to fit doesn't justify their added complexity.

```
ccp_alphas = np.linspace(0.0, 0.01, 25)
train_acc_p, test_acc_p = [], []
for a in ccp_alphas:
    m = DecisionTreeClassifier(random_state=42, ccp_alpha=a)
    m.fit(X_train_t, y_train_t)
    train_acc_p.append(accuracy_score(y_train_t, m.predict(X_train_t)))
    test_acc_p.append(accuracy_score(y_test_t, m.predict(X_test_t)))

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(ccp_alphas, train_acc_p, marker="o", ms=3, label="Train accuracy", color="#3b6ea5")
ax.plot(ccp_alphas, test_acc_p, marker="o", ms=3, label="Test accuracy", color="#c0392b")
ax.set_xlabel("Cost-complexity pruning strength (ccp_alpha)")
ax.set_ylabel("Accuracy")
ax.set_title("Classification Tree: Accuracy vs. Pruning Strength")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("tree_pruning_curve.png")
plt.show()

best_ccp_idx = int(np.argmax(test_acc_p))
print(f"Best ccp_alpha by test accuracy: {ccp_alphas[best_ccp_idx]:.5f}")
```



Best ccp_alpha by test accuracy: 0.00125

2.3 Grid search with time-respecting cross-validation

We combine both levers(max depth and pruning strength) in a small geid search , scored with TimeSeriesSplit so that no fold trains on data from own validation window

```
param_grid = {"max_depth": [2, 3, 4, 5, 6, 8], "ccp_alpha": [0.0, 0.0005, 0.001, 0.002]}
tscv_tree = TimeSeriesSplit(n_splits=5)
tree_grid = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid, cv=tscv_tree, scoring="roc_auc")
tree_grid.fit(X_train_t, y_train_t)
best_tree = tree_grid.best_estimator_

test_pred = best_tree.predict(X_test_t)
test_proba = best_tree.predict_proba(X_test_t)[:, 1]
print("Best params (CV):", tree_grid.best_params_)
print(f"Test accuracy: {accuracy_score(y_test_t, test_pred):.3f}")
print(f"Test ROC-AUC: {roc_auc_score(y_test_t, test_proba):.3f}")
```

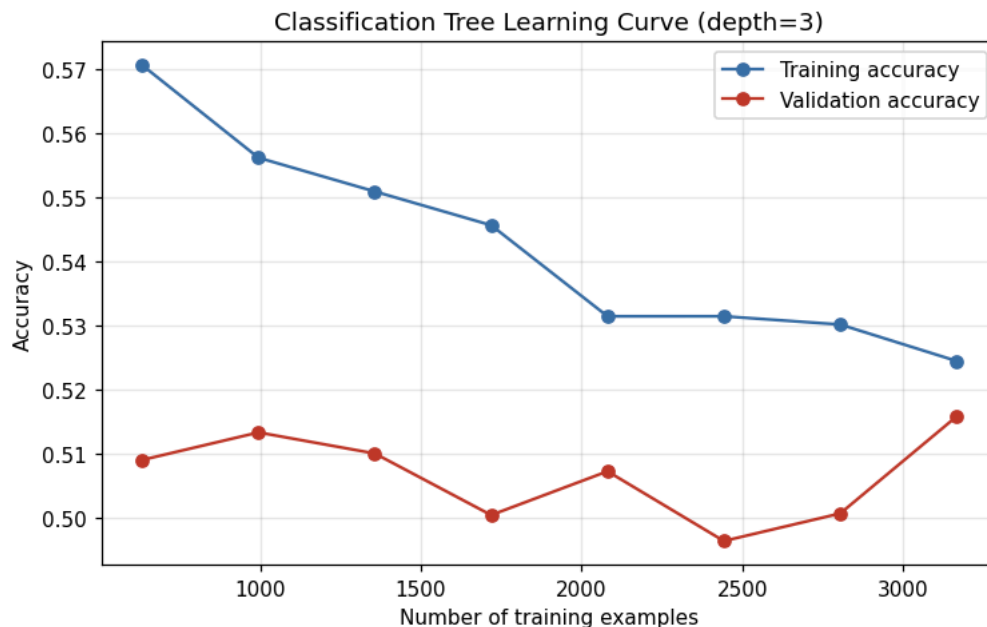
```
Best params (CV): {'ccp_alpha': 0.002, 'max_depth': 3}
Test accuracy: 0.499
Test ROC-AUC: 0.500
```

2.4 Learning curve -dow more data help the tree?

```
train_sizes_t, train_scores_t, val_scores_t = learning_curve(
    DecisionTreeClassifier(max_depth=best_tree.max_depth, random_state=42),
    X_train_t, y_train_t,
    train_sizes=np.linspace(0.2, 1.0, 8),
    cv=5,
    scoring="accuracy",
)

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(train_sizes_t, train_scores_t.mean(axis=1), marker="o", label="Training accuracy", color="#3b6ea5")
ax.plot(train_sizes_t, val_scores_t.mean(axis=1), marker="o", label="Validation accuracy", color="#c0392b")
ax.set_xlabel("Number of training examples")
ax.set_ylabel("Accuracy")
ax.set_title(f"Classification Tree Learning Curve (depth={best_tree.max_depth})")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("tree_learning_curve.png")
plt.show()

print(f"Training accuracy, smallest sample: {train_scores_t.mean(axis=1)[0]:.3f} largest: {train_scores_t.mean(axis=1)[-1]:.3f}")
print(f"Validation accuracy, smallest sample: {val_scores_t.mean(axis=1)[0]:.3f} largest: {val_scores_t.mean(axis=1)[-1]:.3f}")
```



Training accuracy, smallest sample: 0.571 largest: 0.524

Validation accuracy, smallest sample: 0.509 largest: 0.516

Section 3- Model Capacity: LDA (Simple) vs Neural Network (flexible)

We revisit the GWP1 LDA/MLP comparison to make the bias-variance angle explicit: LDA is a single straight decision boundary (low capacity, high bias, low variance by construction), whereas an MLP can bend its boundary arbitrarily as its hidden layer grows (capacity—and variance risk—increases with width). We simulate SPY-like daily price/volume data (matching the GWP1 fallback path) so that the notebook runs standalone.

```
n = 2400
mu_r, sigma_r = 0.08/252, 0.16/np.sqrt(252)
rets = np.random.normal(mu_r, sigma_r, n)
close = 100 * np.exp(np.cumsum(rets))
volume = np.random.lognormal(15.5, 0.4, n)
idx = pd.bdate_range("2015-01-01", periods=n)
px = pd.DataFrame({"Close": close, "Volume": volume}, index=idx)

def rsi(series, window=14):
    delta = series.diff()
    gain = delta.clip(lower=0).rolling(window).mean()
    loss = (-delta.clip(upper=0)).rolling(window).mean()
    rs = gain / loss.replace(0, np.nan)
    return 100 - 100 / (1 + rs)

feat = pd.DataFrame(index=px.index)
ret1 = px["Close"].pct_change()
feat["ret1"] = ret1
feat["ret5"] = px["Close"].pct_change(5)
feat["mom10"] = px["Close"].pct_change(10)
feat["vol10"] = ret1.rolling(10).std()
feat["ma_ratio"] = px["Close"] / px["Close"].rolling(20).mean() - 1
feat["rsi14"] = rsi(px["Close"])
feat["vol_chg"] = px["Volume"].pct_change().clip(-3, 3)
feat["target"] = (ret1.shift(-1) > 0).astype(int)
feat = feat.dropna()

FEATURES = ["ret1", "ret5", "mom10", "vol10", "ma_ratio", "rsi14", "vol_chg"]
Xn = feat[FEATURES].values
yn = feat["target"].values
split = int(len(Xn) * 0.75)
Xn_train, Xn_test = Xn[:split], Xn[split:]
yn_train, yn_test = yn[:split], yn[split:]

scaler_n = StandardScaler()
Xn_train_s = scaler_n.fit_transform(Xn_train)
Xn_test_s = scaler_n.transform(Xn_test)

lda = LinearDiscriminantAnalysis(solver="svd")
lda.fit(Xn_train_s, yn_train)
lda_test_acc = accuracy_score(yn_test, lda.predict(Xn_test_s))
lda_test_auc = roc_auc_score(yn_test, lda.decision_function(Xn_test_s))
```

```
print(f"LDA - test accuracy: {lda_test_acc:.3f}    test ROC-AUC: {lda_test_auc:.3f}")
```

```
LDA - test accuracy: 0.461    test ROC-AUC: 0.460
```

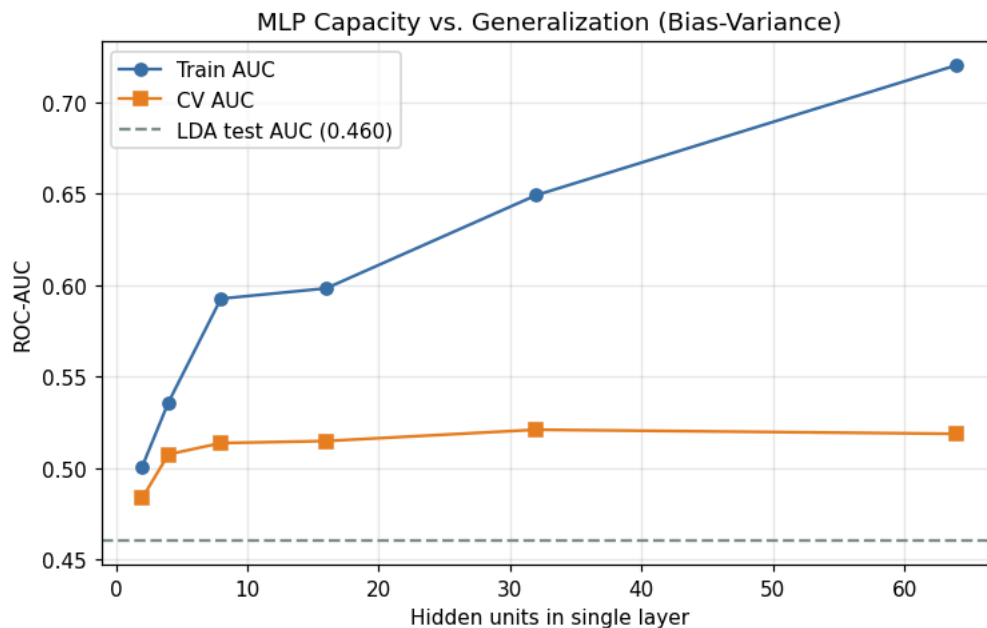
3.1 Validation curve - MLP capacity (hidden layer width) vs generalization

As hidden layer width grows, the network can fit an increasingly intricate boundary. Training AUC should keep climbing; if validation AUC stalls or falls while training AUC keeps rising, that gap is variance/overfitting made visible.

```
widths = [2, 4, 8, 16, 32, 64]
train_auc_curve, val_auc_curve = validation_curve(
    MLPClassifier(max_iter=400, random_state=42, early_stopping=False),
    Xn_train_s, yn_train,
    param_name="hidden_layer_sizes",
    param_range=[(w,) for w in widths],
    cv=TimeSeriesSplit(n_splits=4),
    scoring="roc_auc",
)

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(widths, train_auc_curve.mean(axis=1), marker="o", label="Train AUC", color="#3b6ea5")
ax.plot(widths, val_auc_curve.mean(axis=1), marker="s", label="CV AUC", color="#e67e22")
ax.axhline(lda_test_auc, color="#7f8c8d", linestyle="--", label=f"LDA test AUC ({lda_test_auc:.3f})")
ax.set_xlabel("Hidden units in single layer")
ax.set_ylabel("ROC-AUC")
ax.set_title("MLP Capacity vs. Generalization (Bias-Variance)")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("mlp_validation_curve.png")
plt.show()

print("Train AUC by width:", dict(zip(widths, train_auc_curve.mean(axis=1).round(3))))
print("CV AUC by width:", dict(zip(widths, val_auc_curve.mean(axis=1).round(3))))
```



```
Train AUC by width: {2: 0.501, 4: 0.536, 8: 0.593, 16: 0.598, 32: 0.649, 64: 0.72}
CV AUC by width: {2: 0.483, 4: 0.507, 8: 0.514, 16: 0.515, 32: 0.521, 64: 0.519}
```

3.2 Tuned MLP vs LDA on held out test set

```
best_width = widths[int(np.argmax(val_auc_curve.mean(axis=1)))]
mlp = MLPClassifier(hidden_layer_sizes=(best_width,), alpha=1e-4, learning_rate_init=0.01,
                    max_iter=400, random_state=42)
mlp.fit(Xn_train_s, yn_train)
mlp_test_acc = accuracy_score(yn_test, mlp.predict(Xn_test_s))
mlp_test_auc = roc_auc_score(yn_test, mlp.predict_proba(Xn_test_s)[:, 1])
print(f"Chosen MLP width: {best_width}")
print(f"MLP - test accuracy: {mlp_test_acc:.3f}    test ROC-AUC: {mlp_test_auc:.3f}")
print(f"LDA - test accuracy: {lda_test_acc:.3f}    test ROC-AUC: {lda_test_auc:.3f}")
```

```
Chosen MLP width: 32
MLP - test accuracy: 0.498    test ROC-AUC: 0.494
```

✓ Section 4 - Summary : Bias Variance Tradeoff Across All Three Models